# Interactive Recommendation Agent with Active User Commands


Jiakai Tang[*†]
Gaoling School of Artificial Intelligence, Renmin University of China
Beijing, China
tangjiakai5704@ruc.edu.cn

Wen Chen[*]
Alibaba Group
Beijing, China
chenyu.cw@alibaba-inc.com

Yujie Luo
Alibaba Group
Beijing, China
lyj446765@taobao.com

Xunke Xi
Alibaba Group
Beijing, China
xixunke.xxk@taobao.com

Chao Yi
Alibaba Group
Beijing, China
yichao.yc@alibaba-inc.com

Dian Chen
Alibaba Group
Beijing, China
joshua.cd@alibaba-inc.com

Zhujin Gao
Alibaba Group
Beijing, China
gaozhujin.gzj@taobao.com

Yang Li
Alibaba Group
Beijing, China
ly489842@alibaba-inc.com

Fei Sun
University of Chinese Academy of Sciences
Beijing, China
ofey.sunfei@gmail.com

Xueyang Feng[†]
Gaoling School of Artificial Intelligence, Renmin University of China
Beijing, China
xueyangfeng@ruc.edu.cn

Sunhao Dai[†]
Gaoling School of Artificial Intelligence, Renmin University of China
Beijing, China
sunhaodai@ruc.edu.cn

Xu Chen[✉†]
Gaoling School of Artificial Intelligence, Renmin University of China
Beijing, China
xu.chen@ruc.edu.cn

Jian Wu
Alibaba Group
Beijing, China
joshuawu.wujian@alibaba-inc.com

Yuning Jiang
Alibaba Group
Beijing, China
mengzhu.jyn@alibaba-inc.com

Bo Zheng[✉]
Alibaba Group
Beijing, China
bo.zhengb@alibaba-inc.com



## Abstract

Traditional recommender systems rely on passive feedback mechanisms that limit users to simple choices such as like and dislike. However, these coarse-grained signals fail to capture users' nuanced behavior motivations and intentions. In turn, current systems cannot also distinguish which specific item attributes drive user satisfaction or dissatisfaction, resulting in inaccurate preference modeling. These fundamental limitations create a persistent gap between user intentions and system interpretations, ultimately undermining user satisfaction and harming system effectiveness.

To address these limitations, we introduce the Interactive Recommendation Feed (IRF), a pioneering paradigm that enables natural language commands within mainstream recommendation feeds. Unlike traditional systems that confine users to passive implicit behavioral influence, IRF empowers active explicit control over recommendation policies through real-time linguistic commands. To support this paradigm, we develop RecBot, a dual-agent architecture where a Parser Agent transforms linguistic expressions into structured preferences and a Planner Agent dynamically orchestrates adaptive tool chains for on-the-fly policy adjustment. To enable practical deployment, we employ simulation-augmented knowledge distillation to achieve efficient performance while maintaining strong reasoning capabilities. Through extensive offline and long-term online experiments, RecBot shows significant improvements in both user satisfaction and business outcomes.



## CCS Concepts

• **Information systems** → **Recommender systems**.

## Keywords

Interactive Recommendation, Large Language Models, Agent


[*]Equal contribution.
[†]Also with Beijing Key Laboratory of Research on Large Models and Intelligent Governance, and Engineering Research Center of Next-Generation Intelligent Search and Recommendation, MOE.
[✉]Corresponding author.




This work is licensed under a Creative Commons Attribution 4.0 International License.


*KDD 2026, Jeju Island, Republic of Korea.*


© 2026 Copyright held by the owner/author(s).


ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817619

**ACM Reference Format:**
Jiakai Tang, Wen Chen, Yujie Luo, Xunke Xi, Chao Yi, Dian Chen, Zhujin Gao, Yang Li, Fei Sun, Xueyang Feng, Sunhao Dai, Xu Chen, Jian Wu, Yuning Jiang, and Bo Zheng. 2026. Interactive Recommendation Agent with Active User Commands. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2026), August 9–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770855.3817619

## 1 Introduction

Modern digital ecosystems are saturated with vast amounts of information, making Recommender Systems (RecSys) indispensable components for managing information overload [4, 19], from e-commerce platforms (*e.g.*, Taobao, Amazon) [36, 38, 51, 52] to social media feeds (*e.g.*, TikTok, YouTube) [5, 58]. By filtering and presenting content based on individual user preferences, these systems profoundly shape how users interact with digital content daily. Existing recommender systems typically operate through a *passive feedback loop*, as illustrated in Fig. 1(a). In this paradigm, users are presented with curated recommendation lists and respond through implicit behavioral signals such as clicks, likes, dislikes, or viewing duration. The recommendation algorithms then analyze these signals to infer user preferences and iteratively refine future recommendations, establishing an iterative implicit feedback loop [22, 23, 43].

However, this passive feedback paradigm suffers from fundamental limitations that create a persistent communication gap between user intentions and system interpretations, ultimately undermining recommendation effectiveness. We identify the root causes of this problem from three complementary perspectives: *how users experience interaction constraints, how algorithms interpret ambiguous signals, and how their interplay creates systemic dysfunction.* First, **from the user perspective**, current recommendation interfaces restrict users to rigid interaction mechanisms such as click or star ratings, which cannot adequately capture the nuanced reasoning behind user preferences or identify which specific item attributes drive their satisfaction or dissatisfaction. This design limitation forces users to express complex preferences through vague signals that lose critical information about their actual intentions. Second, **from the algorithm perspective**, faced with such ambiguous and incomplete user feedback, recommendation algorithms resort to indiscriminate preference attribution, treating all item characteristics as equally responsible for user responses. This approach inevitably leads to inaccurate preference modeling that misrepresent user intentions and contribute to filter bubbles that narrow content diversity. Third, **from the ecosystem perspective**, these user expression constraints and algorithmic interpretation flaws interact to create cascading dysfunction beyond their respective impacts. When systems misinterpret signals and users respond to irrelevant recommendations, this creates a communication deadlock where neither party can effectively convey or understand the other's intentions, necessitating fundamental paradigm change rather than incremental improvements. To overcome these limitations, we first propose a novel **Interactive Recommendation Feed (IRF)** product paradigm from a user-centric design philosophy, as illustrated in Fig. 1(b). Unlike traditional interactive recommendation that confine users to passive consumption [6, 33, 57],

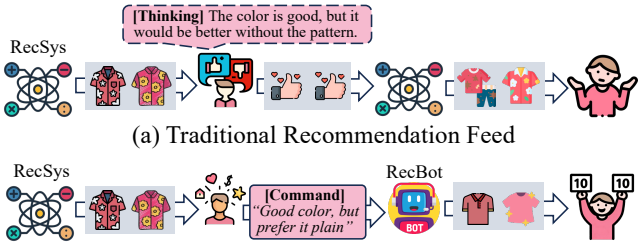



**Figure 1: Comparison between traditional and novel interactive recommendation feeds. (a) Traditional systems rely on constrained and implicit feedback signals (*e.g.*, likes/dislikes), making it difficult to accurately infer users' true intentions. (b) Our interactive paradigm enables free-form natural language commands, where RecBot responds and adjusts recommendation policy on-the-fly based on active user commands.**

IRF introduces an active voice channel enabling users to directly interact with the recommender systems through free-form natural language commands. This transforms the conventional one-way delivery into bidirectional interaction where users can explicitly articulate their requirements and constraints to guide the system's policy adjustment in real-time. Moreover, IRF also differs intrinsically from Conversational Recommender Systems (CRS), which require independent dialogue windows for guided questioning. Instead, IRF seamlessly integrates into mainstream recommendation feeds (e.g., Youtube, Taobao) as a lightweight interface, allowing direct commands without interrupting natural browsing flow. By empowering users with direct algorithmic control, IRF essentially rethinks the human-system relationship, enabling users to proactively shape their content consumption experience rather than being passive recipients of algorithmic decisions. Furthermore, accurately responding to textual user commands requires bridging the gap between open-ended user expressions and actionable recommendation strategies, yet existing interactive methods lack the natural language understanding and reasoning capabilities necessary to parse free-form commands and translate them into command-aware item scoring adjustments. To tackle this problem, we develop ***RecBot***, a large language model-powered multi-agent framework for interactive recommender systems. The framework comprises two core components: First, the **User-Intent-Understanding Agent (*Parser*)** parses natural language descriptions into structured recommendation specifications, extracting both positive and negative user intents while filtering irrelevant information. Second, the **Planning Agent (*Planner*)** then receives these structured preferences and orchestrates customized tool invocation chains, transforming domain-specific requirements into concrete actions that modify item relevance scores. This tool-chaining approach enables the Planner to handle sophisticated user intentions (*e.g.*, positive interest, negative feedback, and interest drift), facilitating on-the-fly policy adaptation that directly influences next recommendation feed. To better support production deployment, RecBot incorporates dynamic memory consolidation for efficient multi-turn interactions and employs knowledge distillation to transfer understanding and reasoning capabilities from powerful teacher models to lightweight student models, achieving scalable and cost-effective deployment.

Our contributions are summarized as follows:

- We introduce the novel Interactive Recommendation Feed (IRF) product paradigm that **breaks the silence of recommender systems** by allowing users to directly communicate with the system through natural language commands, achieving user-centric controllable recommendation.
- We propose RecBot, a multi-agent framework comprising the *Parser* and *Planner* agents that achieve precise intent parsing and on-the-fly command-aware recommendation policy adaptation through automated tool chaining, effectively bridging the gap between free-form user commands and actionable adjustments.
- Extensive offline and online experiments are conducted to comprehensively validate RecBot's effectiveness. Notably, our online deployment demonstrates significant improvements in user engagement, content diversity, and business benefits.

## 2 Problem Setup and Formulation

In this section, we introduce the ***Interactive Recommendation Feed (IRF)*** paradigm, which formulates the recommendation process as a sequential decision-making problem where the system iteratively refines recommendations based on user linguistic commands and behavioral signals. Let $I = \{i_1, i_2, \ldots, i_n\}$ denote the candidate item pool, and $u$ represent a target user. At each interaction round $t \in \{1, 2, \ldots, T\}$, the system presents a recommendation feed $R_t = \{r_t^{(1)}, r_t^{(2)}, \ldots, r_t^{(K)}\} \subset I$ of $K$ items to the user. Upon receiving $R_t$, the user provides linguistic feedback $c_t$. This feedback captures the user's assessment of current recommendations, preferences, or any specific constraints or requirements.

The system maintains a **dual-component user model**: *Explicit Preferences* $P_t$ and *Implicit Preferences* $H_t$. Explicit preferences represent structured knowledge extracted from user linguistic feedback through natural language understanding, formally defined as $P_t = \phi(c_1, c_2, \ldots, c_t)$, where $\phi$ maps a sequence of linguistic feedbacks to structured preference representations. Implicit preferences are captured by the user's historical interaction sequence $H_t = \{i_1, i_2, \ldots, i_m\}$, reflecting long-term behavioral patterns.

At round $t$, the system state is defined as $S_t = \{R_t, c_t, P_t, H_t\}$. The system's action involves selecting the next recommendation feed: $R_{t+1} = \pi(S_t)$. The IRF process operates iteratively: beginning with initial feed $R_0$, the system collects feedback $c_t$, updates explicit preferences $P_t$ while incorporating historical behavior $H_t$, constructs state $S_t$, and generates the next feed $R_{t+1}$. This continues until user satisfaction is achieved satisfy$(u, R_{t+1}) = 1$ or the maximum interaction limit is reached $t \geq T_{\max}$, where $T_{\max}$ represents the threshold for user engagement fatigue beyond which users typically disengage from the system.

## 3 Methodology

This section first provides an overview of our proposed method, then elaborates each core component, and finally presents our multi-agent optimization approach for lightweight deployment.

### 3.1 Overall Architecture

To enable precise responsiveness to user-initiated active commands, we introduce **RecBot**, a multi-agent framework that systematically

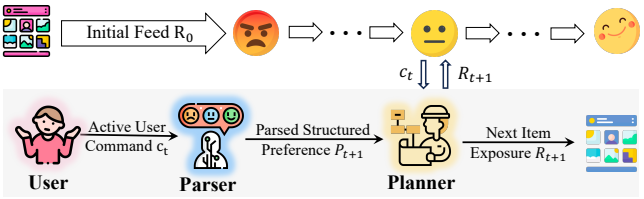



**Figure 2: Overview of the RecBot framework for interactive recommendation. The framework comprises a Parser Agent that transforms user natural language command $c_t$ into structured preferences $P_{t+1}$, and a Planner Agent that orchestrates tool chains to dynamically adjust recommendation policies and generate the next feed $R_{t+1}$.**

decomposes multi-turn user commands, dynamically adjusts recommendation strategies, and modifies subsequent recommendation feeds to progressively satisfy user requirements. The RecBot framework comprises two core intelligent agents coupled with a comprehensive and extensible tool suite, facilitating the transformation from free-form natural language commands to actionable item scoring computations.

As illustrated in Fig. 2, our approach operates through a sequential two-stage process. First, the **User-Intent-Understanding Agent (*Parser*)** transforms user open-domain textual instructions into structured domain-specific recommendation language, parsing both positive and negative user intents while employing a dynamic memory consolidation prompting strategy to maintain efficient multi-turn dialogue state management. Subsequently, the **Planning Agent (Planner)** receives the structured recommendation specifications and orchestrates appropriate tool chain invocations, ultimately curating and presenting the top-$K$ items to users. This decomposition enables RecBot to bridge the semantic gap between diverse commands and concrete recommendation adjustments, facilitating real-time policy adaptation that influences the next feed.

### 3.2 Parser

Real-world user utterances exhibit considerable variability and diversity in both style and semantic content [37, 55], often containing redundant noise such as verbose complaints, irrelevant contextual information. The key challenge lies in transforming free-form user commands into structured recommendation instructions, which is essential for accurately capturing users' proactive intentions. Inspired by the robust contextual reasoning capabilities of large language models [21, 27, 34, 47], we design a **User Intent Understanding Agent (*Parser*)** that bridges the semantic gap between diverse user expressions and actionable recommendation specifications. The *Parser* exploits the inherent ability of large language models to handle linguistic variability and extract meaningful signals from noisy textual input, enabling precise intent understanding.

*3.2.1* ***Structured Command Parsing.*** Given the previous recommendation feed $R_t$ presented to the user, user linguistic feedback $c_t$ in response to $R_t$, and historical preference memory $P_t$, the *Parser* performs the transformation:

$$\mathcal{P} : (R_t, c_t, P_t) \rightarrow P_{t+1}, \tag{1}$$

where $\mathcal{P}$ denotes the parsing function that maps the input tuple to updated preference specifications $P_{t+1}$. The updated preference specification $P_{t+1}$ decomposes into two orthogonal dimensions based on user sentiment orientation:

$$P_{t+1} = \{P^+_{t+1}, P^-_{t+1}\}. \tag{2}$$

where $P^+_{t+1}$ represents **positive preferences** (item attributes the user shows interest in) and $P^-_{t+1}$ represents **negative preferences** (item attributes the user wants to exclude). This bidirectional decomposition enables better modeling of user intentions by capturing both attraction and aversion signals from natural language feedback. Furthermore, each preference dimension further categorizes into constraint types based on enforcement strictness:

$$P^+_{t+1} = \{C^{+,\text{hard}}_{t+1}, C^{+,\text{soft}}_{t+1}\}, \quad P^-_{t+1} = \{C^{-,\text{hard}}_{t+1}, C^{-,\text{soft}}_{t+1}\}, \tag{3}$$

where $C^{\text{hard}}$ represents **strict rule-based constraints** that can be deterministically verified through direct attribute matching (*e.g.*, "under \$50"), and $C^{\text{soft}}$ denotes **flexible semantic-based inclinations** that require nuanced understanding of user's subjective interests (*e.g.*, "prefer romantic movies").

*3.2.2* ***Dynamic Memory Consolidation.*** Multi-turn interactions are crucial for interactive recommender systems as they allow users to iteratively refine their preferences and guide the system toward more in-depth understanding of their evolving interests. However, multi-turn interactions introduce extra computational and semantic challenges in maintaining consistent user preference understanding across extended dialogue sequences. Naive approaches that accumulate extensive conversation histories lead to computational overhead and semantic drift, while oversimplified state management fails to capture evolving user intentions accurately. To this end, we design a **Dynamic Memory Consolidation Strategy** that maintains preference coherence across interaction rounds while ensuring computational efficiency through principled state management rather than exhaustive history retention.

In specific, the consolidation mechanism operates through three strategic update prompting principles that guide the Parser's preference synthesis process:

**(1) Preservation Principle**: When current feedback $c_t$ indicates satisfaction with existing recommendations or provides neutral commentary, historical preferences $P_t$ are preserved unchanged to maintain preference stability: $P_{t+1} = P_t$.

**(2) Integration Principle**: When feedback introduces compatible new preferences that complement existing memory, these preferences are integrated through structured merging operations:

$$P^+_{t+1} = P^+_t \oplus \text{Extract}^+(c_t), \qquad P^-_{t+1} = P^-_t \oplus \text{Extract}^-(c_t),$$

where $\oplus$ denotes the structured integration operator that handles both hard constraints and soft preferences separately, and $\text{Extract}^{\pm}(c_t)$ extracts positive and negative preference signals from current user command $c_t$ respectively.

**(3) Resolution Principle**: When feedback explicitly contradicts established preferences, the Parser employs linguistic cue analysis to identify change indicators (e.g., "instead of," "no longer interested in," "different from before") and performs targeted preference updates while preserving non-conflicting knowledge: $P_{t+1} = \text{Resolve}(P_t, c_t, R_t)$.

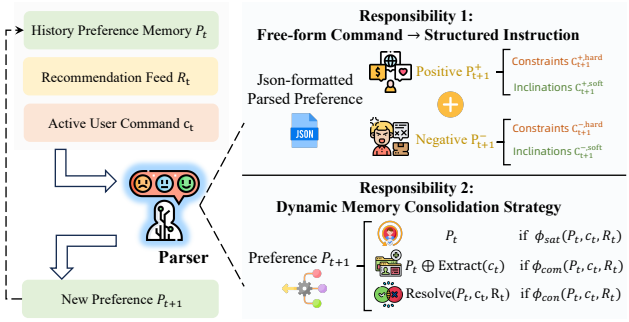



**Figure 3: Illustration of the *Parser* for user intent understanding. The *Parser* integrates history preference memory $P_t$, current recommendation feed $R_t$, and active user command $c_t$ to generate new preference representation $P_{t+1}$ through structured parsing and dynamic memory consolidation.**

The consolidation process is formalized through a **context-aware decision function** that dynamically selects the appropriate update strategy based on the interplay between historical preferences $P_t$, current feedback $c_t$, and recommendation context $R_t$:

$$P_{t+1} = \begin{cases} P_t & \text{if } \phi_{\text{sat}}(c_t, R_t), \\ P_t \oplus \text{Extract}(c_t) & \text{if } \phi_{\text{com}}(P_t, c_t, R_t), \\ \text{Resolve}(P_t, c_t, R_t) & \text{if } \phi_{\text{con}}(P_t, c_t, R_t), \end{cases}$$

where $\phi_{\text{sat}}$, $\phi_{\text{com}}$, and $\phi_{\text{con}}$ represent context-aware analysis functions that respectively detect user satisfaction states, preference compatibility, and preference conflicts.

## 3.3 Planner

Upon receiving the structured recommendation specifications $P_{t+1}$ from the *Parser*, the next task is to translate these explicit user preferences into concrete recommendation policy adjustments. To address this, we introduce the **Planning Agent (*Planner*)**, which orchestrates adaptive tool chain invocations to dynamically modify item scoring mechanisms based on user commands.

The *Planner* operates through principled reasoning over a comprehensive **recommendation domain toolset**, enabling flexible adaptation to diverse user requirements. Given structured preferences $P_{t+1} = \{P^+_{t+1}, P^-_{t+1}\}$ and user implicit preferences $H_t$, the *Planner* performs the mapping:

$$\mathcal{A} : (P_{t+1}, H_t, I) \rightarrow S_{t+1}, \tag{4}$$

where $\mathcal{A}$ denotes the planning function that transforms preference specifications and candidate item pool $I$ into updated item scoring $S_{t+1} = \{s_1, s_2, \ldots, s_n\}$, enabling top-$K$ item selection for the subsequent recommendation feed $R_{t+1}$.

*3.3.1* ***Recommendation Domain Toolset.*** The *Planner* uses a modular and extensible toolset, each designed to handle specific aspects of user preference satisfaction through targeted item scoring. The core toolset comprises four complementary components: (i) ***Filter*** for hard constraint enforcement, (ii) ***Matcher*** for positive preference alignment, (iii) ***Attenuator*** for negative feedback incorporation, and (iv) ***Aggregator*** for comprehensive score integration. In the following, we detail the specific functionality of each tool.

***Filter Tool.*** The *Filter* enforces hard constraint satisfaction by both selecting items that meet positive requirements and eliminating items that violate negative constraints. Given hard constraints $C^{+,\text{hard}}_{t+1}$ and $C^{-,\text{hard}}_{t+1}$ from $P_{t+1}$, the filter adopts binary selection over the candidate item pool $I$ to produce a refined subset $I'$:

$$I' = \{i \in I : C^+(i, C^{+,\text{hard}}_{t+1}) = 1 \wedge C^-(i, C^{-,\text{hard}}_{t+1}) = 0\}, \tag{5}$$

where $C^+(i, C^{+,\text{hard}}_{t+1})$ evaluates whether item $i$ satisfies all positive hard constraints, and $C^-(i, C^{-,\text{hard}}_{t+1})$ determines whether item $i$ violates any negative hard constraints. Items excluded from $I'$ are effectively assigned a score of $-\infty$, ensuring they are not recommended in the subsequent feed.

This filtering mechanism enables efficient candidate space pruning while enforcing strict user requirements. For instance, when users specify temporal constraints (*e.g.*, "movies released before 2020") or budget limitations (*e.g.*, "no luxury products above $500"), the *Filter* precisely narrows the search space according to these explicit boundaries. The resulting filtered candidate set $I'$ serves as the refined input for downstream scoring tools, namely the *Matcher* and *Attenuator*, ensuring computational efficiency while maintaining strict adherence to user-specified requirements.

***Matcher Tool.*** The *Matcher* tool computes positive relevance scores based on user positive intentions through a two-path architecture that combines semantic understanding with personalized collaborative knowledge.

**(1) *Semantic Relevance Path:*** Positive intentions $P^+_{t+1}$ are first transformed into structured natural language descriptions following the format: "attribute$_1$:[value, ...], attribute$_2$:[value, ...]." The semantic component leverages pre-trained embedding model (*e.g.*, BGE [46], Sentence-BERT [29]) to compute semantic affinity between item descriptions and positive intent representations:

$$s_{\text{sem}}(i, P^+_{t+1}) = \text{sim}(\mathbf{e}_{\text{item}}(i), \mathbf{e}_{\text{intent}}(P^+_{t+1})) \tag{6}$$

where $\mathbf{e}_{\text{item}}(i)$ and $\mathbf{e}_{\text{intent}}(P^+_{t+1})$ denote the embedding representations of item $i$'s description and formatted positive intentions, respectively, and $\text{sim}(\cdot,\cdot)$ indicates cosine similarity.

**(2) *Active-Intent-Aware Collaborative Path:*** We design an *Active-Intent-Aware (AIA)* sequential recommender that leverages user positive preference as queries to extract intent-relevant patterns from multimodal historical interactions.

Specifically, we first encodes multimodal features of item $i$ into modal-specific representations $\mathbf{h}^i_m \in \mathbb{R}^d$ via encoders $f_m(\cdot)$, where $m$ denotes modality type (*e.g.*, text and image), then fuses them into unified representation: $\mathbf{h}^i_{\text{fused}} = \text{Linear}([\mathbf{h}^i_{\text{text}} \oplus \mathbf{h}^i_{\text{image}} \oplus \ldots]) \in \mathbb{R}^d$, where $\text{Linear}(\cdot)$ performs dimension reduction from concatenated multimodal features to unified $d$-dimensional representations. After that, sequential encoder is used to extract temporal patterns from user historical interactions $H_t$: $\mathbf{H}_{\text{fused}} = \text{SeqEnc}([\mathbf{h}^{i_1}_{\text{fused}}, \ldots, \mathbf{h}^{i_{|H_t|}}_{\text{fused}}]) \in \mathbb{R}^{|H_t| \times d}$, where SeqEnc employs *Multi-Head Self-Attention (MHSA)* mechanisms. Furthermore, to capture intent-aware collaborative patterns, we employ *Multi-Head Cross-Attention (MHCA)* where user intent serves as the query while fused multimodal sequences act as keys and values, which is fomulated as follows:

$$s_{\text{aia}}(i_{\text{cand}}, P^+_{t+1}, H_t) = \text{MHCA}(\mathbf{h}_{\text{intent}}, \mathbf{H}_{\text{fused}}, \mathbf{H}_{\text{fused}}) \cdot \mathbf{h}^{i_{\text{cand}}},$$

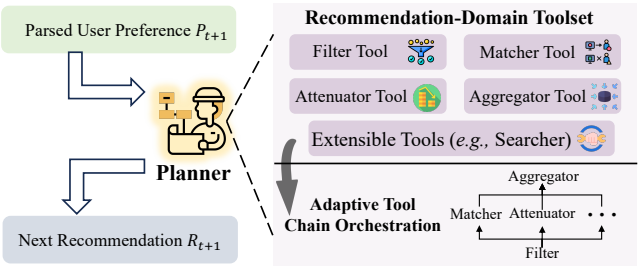



**Figure 4: Illustration of the *Planner* for on-the-fly recommendation policy adaptation. The *Planner* dynamically constructs optimal tool invocation sequences based on parsed user preferences $P_{t+1}$ to compute updated item scores $s_{\text{final}}$ for next recommendation feed $R_{t+1}$.**

where $\mathbf{h}_{\text{intent}} = f_{\text{text}}(\mathbf{e}_{\text{intent}}(P^+_{t+1}))$ represents encoded user intent, and $i_{\text{cand}} \in I'$ denotes a candidate item from the filtered set.

The final positive relevance score integrates both semantic and collaborative components:

$$s_{\text{match}}(i) = \alpha \cdot s_{\text{sem}}(i, P^+_{t+1}) + (1-\alpha) \cdot s_{\text{aia}}(i, P^+_{t+1}, H_t), \tag{7}$$

where $\alpha \in [0,1]$ balances semantic and collaborative contributions.

***Attenuator Tool.*** The *Attenuator* computes negative relevance penalties based on user negative intentions. Similar to the semantic path in *Matcher*, negative intentions $P^-_{t+1}$ are converted to structured descriptions and semantic similarity scores are computed:

$$s_{\text{atten}}(i) = -\beta \cdot \text{sim}(\mathbf{e}_{\text{item}}(i), \mathbf{e}_{\text{intent}}(P^-_{t+1})), \tag{8}$$

where $\beta > 0$ controls the attenuation strength, and the negative sign ensures that items semantically similar to negative intentions receive penalty scores, thereby reducing their likelihood of being recommended in the next feed.

***Aggregator Tool.*** The *Aggregator* tool synthesizes scores from all preceding tools to produce final item rankings:

$$s_{\text{final}}(i) = s_{\text{match}}(i) + s_{\text{atten}}(i), \tag{9}$$

Finally, the top-$K$ items based on final scores $s_{\text{final}}(i)$ constitute the next recommendation feed $R_{t+1}$.

*3.3.2* ***Adaptive Tool Chain Orchestration.*** The *Planner* employs context-aware reasoning to dynamically construct optimal tool invocation sequences based on parsed user preferences $P_{t+1}$. The orchestration operates through two coordinated phases: **(1) Strategic Tool Selection:** The *Planner* analyzes $P_{t+1}$ to determine necessary tool activations: hard constraints ($C^{+,\text{hard}}_{t+1}$, $C^{-,\text{hard}}_{t+1}$) trigger prioritized *Filter* activation, positive intentions ($C^{+,\text{soft}}_{t+1}$) activate the *Matcher*, while negative feedback ($C^{-,\text{soft}}_{t+1}$) necessitates *Attenuator* deployment. The *Planner* adaptively scales tool complexity, deploying selective subsets for focused feedback or full sequences (*Filter* → *Matcher* & *Attenuator* → *Aggregator*) for multi-dimensional requirements. **(2) Execution Coordination:** The *Planner* manages seamless information flow between tools, ensuring filtered candidate set $I'$ constrains all scoring computations within valid item space, while the *Aggregator* synthesizes multi-tool outputs through weighted combination for final item ranking.

**Table 1: Comparison of Different Interactive Recommendation Agent Frameworks. The comparison criteria are categorized into three aspects: Agent Capabilities, Interaction Features, and Practicality.**

| Capability | InteRecAgent [16] | RecBench+ [15] | InstructAgent [48] | GOMMIR [45] | RecBot (Ours) |
|---|---|---|---|---|---|
| Collaborative Knowledge | ✓ | ✗ | ✗ | ✓ | ✓ |
| Tool Invocation | ✓ | ✗ | ✓ | ✗ | ✓ |
| Agent-Tuning | ✓ | ✗ | ✗ | ✓ | ✓ |
| Memory | ✓ | ✗ | ✓ | ✓ | ✓ |
| Multi-Modal | ✗ | ✗ | ✗ | ✓ | ✓ |
| Multi-turn Interaction | ✓ | ✗ | ✗ | ✓ | ✓ |
| Complex User Command | ✗ | ✗ | ✗ | ✗ | ✓ |
| Industrial Deployment | ✗ | ✗ | ✗ | ✗ | ✓ |

## 3.4 Multi-Agent Optimization

The *Parser* and *Planner* agents face complex personalized reasoning tasks involving ambiguous user command parsing and adaptive tool orchestration. While closed-source models (*e.g.*, GPT-4.1 [1]) excel at these tasks, they face deployment constraints including prohibitive costs and privacy concerns. To achieve lightweight deployment, we introduce a **Multi-Agent Optimization** framework that transfers knowledge from powerful teacher models to cost-effective student models through simulation-augmented distillation.

***Simulation-Augmented Knowledge Distillation.*** We leverage role-playing capabilities [10, 56] to construct a simulation environment where synthetic **User Simulation Agent** $\mathcal{U}_{\text{sim}}$ and **Teacher RecBot** $\mathcal{R}_{\text{teacher}}$ engage in multi-turn interactions. Given target item $i_{\text{target}}$ and recommendation feed $R_t$, the user agent generates contextual feedback:

$$c_t^{\text{sim}} = \mathcal{U}_{\text{sim}}(R_t, i_{\text{target}}, \mathcal{G}_{\text{persona}}),$$

where $\mathcal{G}_{\text{persona}}$ defines user behavioral characteristics. The teacher processes this feedback to generate structured preferences and tool chains, creating diverse interaction trajectories.

***Optimization Objective.*** From simulation trajectories, we extract training samples for both agents: Parser samples $(R_t, c_t^{\text{sim}}, P_t) \rightarrow P_{t+1}^{\text{sim}}$ and Planner samples $P_{t+1}^{\text{sim}} \rightarrow \mathcal{T}_t^{\text{sim}}$. We optimize both agents jointly through Supervised Fine-Tuning on mixed training data $\mathcal{D}_{\text{Mixed}} = \mathcal{D}_{\text{Parser}} \cup \mathcal{D}_{\text{Planner}}$ using Next-Token Prediction:

$$\mathcal{L}(\theta) = \sum_{(x,y)\in\mathcal{D}_{\text{Mixed}}} \sum_{j=1}^{|y|} -\log P_\theta(y_j | x, y_{<j}) \tag{10}$$

This unified training enables cost-effective deployment while maintaining functional modularity.

## 3.5 Discussion

*3.5.1 Comparison of RecBot and Existing Interactive Recommendation Agents.* As summarized in Table 1, we compare RecBot against existing interactive recommendation agents across three dimensions. RecBot demonstrates multi-aspect advancement: **(1) Agent Capabilities:** RecBot features five key advances: (i) *Collaborative Knowledge* integrating historical behaviors with explicit commands; (ii) *Tool Invocation* enabling flexible adaptation through modular interfaces; (iii) *Agent-Tuning* achieving production-ready performance via knowledge distillation; (iv) *Memory* maintaining preference coherence through dynamic consolidation; (v) *Multi-Modal* processing heterogeneous item representations across modalities. **(2) Interaction Features:** RecBot handles complex multi-turn dialogues and sophisticated commands (*e.g.*, mixed user feedback, interest shifts), addressing limitations of prior single-turn approaches. **(3) Practical Impact:** RecBot achieves successful industrial deployment with validated business impact, while existing approaches remain theoretical prototypes.

*3.5.2 Interactive Recommendation Feed vs. Conversational Recommendation.* The IRF paradigm differs fundamentally from conversational recommender systems across two dimensions: **(1) Usage Scenario:** CRS address goal-directed search tasks where users possess well-defined information needs and explicit purchase intentions. IRF, conversely, facilitates exploratory browsing scenarios where users engage in open-ended discovery, with preferences emerging and evolving through iterative system interactions. **(2) Preference Elicitation:** CRS employ interrogative approaches that strategically extract user preferences via predefined conversational schemas. IRF adopts a reactive paradigm where users provide contextually-anchored feedback through natural language responses to specific recommendation outputs, enabling preference articulation directly grounded in presented recommendation feed.

# 4 Experiments

## 4.1 Experimental Setup

*4.1.1* ***Datasets and Interactive Scenarios.*** We conduct experiments on three recommendation datasets: **Amazon Books**, **MovieLens**, and **Taobao**. Each dataset incorporates both hard constraints (*e.g.*, price, release date) and soft conditions (*e.g.*, categories, genres) for interactive recommendation guidance. We design three interaction scenarios of increasing complexity: **Single-Round Interaction (SR)** for users with clear preferences, **Multi-Round Interaction (MR)** for exploratory users with iterative refinement, and **Multi-Round Interaction with Interest Drift (MRID)** for users exhibiting preference evolution. Multi-round scenarios are limited to 5 rounds with $K = 5$ recommendations per round.

*4.1.2* ***Evaluation Metrics.*** For offline experiments, we adopt ranking metrics Recall@N and NDCG@N where $N \in \{10, 20, 50\}$ to

**Table 2: Performance comparison of different methods on Single-Round (SR) interaction scenarios. We report results based on Recall@K (R@K), Condition Satisfaction Rate@K (C@K), and Pass Rate (PR) metrics. Best results are highlighted in bold.**

| | Amazon | | | | | MovieLens | | | | | Taobao | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Method** | **R@10** | **R@20** | **C@10** | **C@20** | **PR** | **R@10** | **R@20** | **C@10** | **C@20** | **PR** | **R@10** | **R@20** | **C@10** | **C@20** | **PR** |
| SASRec | 0.0098 | 0.0163 | 65.81 | 69.08 | 0.76 | 0.0236 | 0.0428 | 57.55 | 62.15 | 1.61 | 0.0267 | 0.0400 | 37.75 | 40.36 | 1.67 |
| BERT4Rec | 0.0076 | 0.0109 | 64.84 | 69.40 | 0.65 | 0.0364 | 0.0493 | 57.44 | 61.99 | 1.71 | 0.0210 | 0.0340 | 36.56 | 39.74 | 1.20 |
| MoRec | 0.0316 | 0.0457 | 73.49 | 77.65 | 2.29 | 0.0642 | 0.1103 | 60.65 | 66.43 | 3.00 | 0.1421 | 0.1922 | 45.31 | 49.13 | 9.48 |
| UniSRec | 0.0370 | 0.0544 | 73.87 | 77.65 | 2.18 | 0.0664 | 0.1338 | 60.28 | 66.60 | 4.07 | 0.1535 | 0.2095 | 45.16 | 49.14 | 9.41 |
| BM25 | 0.0283 | 0.0370 | 80.51 | 85.96 | 1.31 | 0.0257 | 0.0535 | 60.81 | 66.86 | 1.28 | 0.0000 | 0.0000 | 28.10 | 34.00 | 0.00 |
| BGE | 0.0598 | 0.1012 | 92.76 | 95.19 | 3.16 | 0.1370 | 0.2077 | 77.25 | 85.06 | 7.49 | 0.2025 | 0.2753 | 82.60 | 86.35 | 14.25 |
| GOMMIR | 0.0011 | 0.0022 | 53.14 | 60.89 | 0.00 | 0.0021 | 0.0043 | 43.36 | 43.36 | 0.11 | 0.0003 | 0.0003 | 17.24 | 19.70 | 0.03 |
| InteRecAgent | 0.0609 | 0.1023 | 81.98 | 84.21 | 3.81 | 0.1103 | 0.1959 | 66.81 | 73.45 | 6.64 | 0.2122 | 0.2673 | 57.18 | 60.74 | 15.72 |
| Intruct2Agent | 0.0033 | 0.0054 | 67.02 | 72.42 | 0.00 | 0.0214 | 0.0460 | 54.12 | 59.37 | 1.28 | 0.0230 | 0.0377 | 34.15 | 41.11 | 1.20 |
| RecBot-Qwen (Orig.) | 0.2078 | 0.2851 | 92.12 | 93.70 | 14.04 | 0.3073 | 0.4058 | 80.57 | 83.99 | 23.13 | 0.3797 | 0.4668 | 81.18 | 84.59 | 29.56 |
| RecBot-Qwen (Align.) | 0.2198 | 0.3101 | 94.92 | 96.13 | 14.58 | 0.3383 | 0.4208 | 81.10 | 84.69 | 25.70 | 0.4531 | 0.5402 | 86.12 | 88.39 | 35.37 |
| RecBot-GPT | **0.2459** | **0.3547** | **97.63** | **98.47** | **16.76** | **0.4293** | **0.5161** | **88.81** | **91.22** | **32.76** | **0.4918** | **0.5879** | **89.40** | **91.58** | **39.31** |

assess recommendation quality, alongside interactive-specific metrics: Condition Satisfaction Rate (CSR@N) measuring attribute-level coverage, Pass Rate (PR) indicating successful target delivery, and Average Rounds (AR) quantifying interaction efficiency. For conciseness, we primarily present R@10/20, CSR, PR, and AR metrics in the main results. For online experiments, we evaluate *User Experience Metrics* including Negative Feedback Frequency (NFF), Exposed Item Category Diversity (EICD), and Clicked Item Category Diversity (CICD), alongside *Business Utility Metrics* including Page Views (PV), Add-to-Cart (ATC), and Gross Merchandise Volume (GMV).

*4.1.3* ***Baselines.*** We compare RecBot against three categories of methods: **(1) Traditional Sequential Recommendation** including SASRec [18], BERT4Rec [35], MoRec [53], and UniSRec [11]; **(2) Command-Aware Recommendation** including BM25 [30] and BGE [46]; **(3) Interactive Recommendation Agents** including GOMMIR [45], InteRecAgent [16], and Instruct2Agent [48]. We evaluate three RecBot variants: RecBot-Qwen (Orig.), RecBot-Qwen (Align.), and RecBot-GPT. For more detailed experimental configurations, please refer to the Appendix A.

*4.1.4* ***Implementation Details.*** For multi-agent optimization, we employ Qwen3-14B-Instruct [50] as our backbone large language model, implemented using OpenRLHF [14] and PyTorch [26] frameworks. We adopt LoRA [13] for Parameter-Efficient Fine-Tuning (PEFT) with rank=32, alpha=32, dropout=0.1, training for 3 epochs with learning rate $5e^{-6}$ and batch size 256. User historical interaction sequences are truncated to a maximum length of 50 items, and BGE [46] serves as the pre-trained embedding model for all semantic computations. The simulation-augmented knowledge distillation leverages GPT-4.1 [1] as the teacher model to generate training trajectories for fine-tuning the student model.

## 4.2 Offline Experiments

*4.2.1 Overall Performance.* The performance comparison results across Single-Round (SR), Multi-Round (MR), and Multi-Round with Interest Drift (MRID) interaction scenarios are presented in Tables 2 and 3. These results reveal several compelling findings: (1) **Multimodal superiority over ID-based modeling:** Multimodal approaches (MoRec, UniSRec) consistently outperform ID-based methods (SASRec, BERT4Rec) by effectively capturing fine-grained sequence patterns. For instance, UniSRec achieves 0.0370 Recall@10 on Amazon SR, significantly surpassing SASRec's 0.0098, demonstrating the importance of rich content representations. (2) **Command-aware methods outperform implicit modeling:** BGE consistently outperforms BM25 due to superior semantic comprehension, achieving 0.1370 vs. 0.0257 Recall@10 on MovieLens SR. This highlights the advantage of explicit linguistic feedback over pure implicit behavioral signals. (3) **Limitations of existing interactive agents:** GOMMIR suffers from naive encoding approaches, while InteRecAgent and InstructAgent lack fine-grained command parsing and domain-specific collaborative knowledge, limiting their performance in complex scenarios. (4) **RecBot achieves state-of-the-art performance:** Our framework excels across all datasets and metrics, particularly in multi-round scenarios. On Taobao MR, RecBot-GPT achieves 41.14% pass rate with 4.28 average rounds versus InteRecAgent's 18.42% and 5.08 rounds. Notably, our knowledge distillation enables RecBot-Qwen (Align.) to surpass its GPT-4.1 teacher on MovieLens MRID (0.3940 vs. 0.3158 Recall@10), demonstrating that focused knowledge transfer can unlock latent capabilities in smaller models [12, 49].

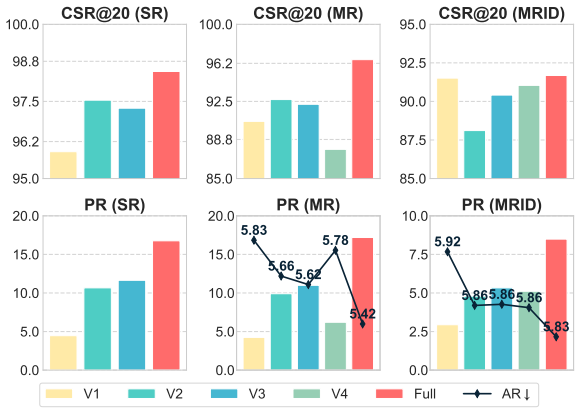



**Figure 5: Offline ablation study results on Amazon dataset. All numerical values on axes correspond to percentages.**

*4.2.2 Ablation Study.* To evaluate each component's contribution, we conduct ablation experiments comparing five RecBot variants: **(1) V1** uses only Semantic Relevance Path; **(2) V2** employs only Active-Intent-Aware Collaborative Path; **(3) V3** combines complete

**Table 3: Performance comparison on Multi-Round (MR) and Multi-Round with Interest Drift (MRID) interaction scenarios.**

| Method | Multi-Round (MR) | | | | | | Multi-Round with Interest Drift (MRID) | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | R@10↑ | R@20↑ | C@10↑ | C@20↑ | PR↑ | AR↓ | R@10↑ | R@20↑ | C@10↑ | C@20↑ | PR↑ | AR↓ |
| **Amazon** | | | | | | | | | | | | |
| BM25 | 0.0381 | 0.0479 | 77.37% | 83.44% | 3.70% | 5.8194 | 0.0424 | 0.0544 | 78.55% | 84.28% | 3.16% | 5.8564 |
| BGE | 0.0609 | 0.0740 | 87.29% | 89.84% | 5.22% | 5.7661 | 0.0316 | 0.0392 | 80.65% | 85.14% | 2.72% | 5.8531 |
| GOMMIR | 0.0000 | 0.0011 | 52.20% | 64.01% | 0.00% | 6.0000 | 0.0011 | 0.0011 | 50.08% | 60.05% | 0.00% | 6.0000 |
| InteRecAgent | 0.0533 | 0.0849 | 78.89% | 81.77% | 3.70% | 5.8085 | 0.0511 | 0.0783 | 76.48% | 79.67% | 3.26% | 5.8335 |
| Intruct2Agent | 0.0250 | 0.0283 | 66.69% | 71.98% | 2.50% | 5.8596 | 0.0239 | 0.0250 | 65.26% | 70.97% | 2.29% | 5.8640 |
| RecBot-Qwen (Orig.) | 0.1632 | 0.1904 | 82.30% | 83.99% | 12.30% | 5.6126 | 0.0958 | 0.1360 | 77.54% | 81.01% | 6.86% | 5.8466 |
| RecBot-Qwen (Align.) | 0.1893 | 0.2416 | 84.21% | 86.20% | 15.02% | 5.5234 | 0.1056 | 0.1371 | 77.84% | 80.73% | 8.38% | 5.8292 |
| RecBot-GPT | **0.2296** | **0.3199** | **95.20%** | **96.56%** | **17.19%** | **5.4244** | **0.1164** | **0.1513** | **89.13%** | **91.68%** | **8.49%** | **5.8324** |
| **MovieLens** | | | | | | | | | | | | |
| BM25 | 0.0621 | 0.0931 | 63.28% | 68.47% | 5.57% | 5.7099 | 0.0600 | 0.0942 | 62.31% | 69.49% | 5.03% | 5.7195 |
| BGE | 0.1146 | 0.1745 | 71.63% | 79.76% | 9.53% | 5.5675 | 0.0974 | 0.1360 | 66.33% | 73.77% | 6.64% | 5.6895 |
| GOMMIR | 0.0043 | 0.0064 | 43.47% | 43.47% | 0.32% | 5.9818 | 0.0043 | 0.0064 | 43.47% | 43.47% | 0.32% | 5.9818 |
| InteRecAgent | 0.1081 | 0.1820 | 65.42% | 71.31% | 7.49% | 5.6006 | 0.1039 | 0.1788 | 65.90% | 71.73% | 7.17% | 5.6702 |
| Intruct2Agent | 0.0557 | 0.0749 | 55.09% | 60.55% | 5.03% | 5.7173 | 0.0578 | 0.0696 | 54.93% | 58.94% | 5.35% | 5.7099 |
| RecBot-Qwen (Orig.) | 0.3812 | 0.4625 | 84.21% | 86.30% | 34.05% | 4.9261 | 0.2955 | 0.3683 | 81.96% | 84.53% | 24.52% | 5.4111 |
| RecBot-Qwen (Align.) | **0.4315** | **0.5021** | **87.53%** | **89.61%** | **38.12%** | **4.7837** | **0.3940** | **0.4582** | **84.05%** | **85.92%** | **33.51%** | **5.2591** |
| RecBot-GPT | 0.4036 | 0.4829 | 86.13% | 88.28% | 35.01% | 4.8298 | 0.3158 | 0.4111 | 81.48% | 84.85% | 26.02% | 5.3951 |
| **Taobao** | | | | | | | | | | | | |
| BM25 | 0.0941 | 0.0941 | 35.01% | 40.33% | 9.41% | 5.4354 | 0.0941 | 0.0941 | 36.25% | 41.32% | 9.41% | 5.4354 |
| BGE | 0.1919 | 0.2259 | 64.11% | 68.30% | 17.18% | 5.1235 | 0.0998 | 0.1024 | 37.41% | 42.32% | 9.68% | 5.4308 |
| GOMMIR | 0.0000 | 0.0000 | 15.04% | 19.07% | 0.00% | 6.0000 | 0.0000 | 0.0000 | 14.93% | 19.25% | 0.00% | 6.0000 |
| InteRecAgent | 0.2166 | 0.2619 | 53.26% | 57.21% | 18.42% | 5.0791 | 0.2089 | 0.2526 | 51.94% | 55.52% | 16.85% | 5.2155 |
| Intruct2Agent | 0.1061 | 0.1091 | 33.85% | 38.67% | 10.44% | 5.3971 | 0.0948 | 0.0948 | 23.55% | 26.45% | 9.44% | 5.4348 |
| RecBot-Qwen (Orig.) | 0.3967 | 0.4518 | 82.44% | 84.83% | 35.14% | 4.5122 | 0.2396 | 0.2736 | 56.50% | 59.14% | 20.42% | 5.2456 |
| RecBot-Qwen (Align.) | 0.4238 | 0.4735 | 76.94% | 79.35% | 38.47% | 4.3827 | **0.2964** | **0.3432** | 65.00% | 67.80% | **25.60%** | 5.1906 |
| RecBot-GPT | **0.4618** | **0.5305** | **84.86%** | **87.32%** | **41.14%** | **4.2809** | 0.2826 | 0.3377 | **72.18%** | **75.34%** | 22.92% | **5.1869** |

**Table 4: Online average performance improvement of RecBot over the baseline model during the three-month A/B testing.**

| NFF↓ | EICD↑ | CICD↑ | PV↑ | ATC↑ | GMV↑ |
|---|---|---|---|---|---|
| **-0.71%** | **+0.88%** | **+1.44%** | **+0.56%** | **+1.28%** | **+1.40%** |

Matcher with Attenuator Tool; **(4) V4** implements simplified BGE-based semantic approach; and **(5) Full** represents complete RecBot with entire tool suite. All experiments are conducted using GPT-4.1.

As shown in Figure 5, Full RecBot consistently achieves superior performance, particularly in MR tasks with higher pass rates and fewer interaction rounds. V1 exhibits weakest performance due to reliance solely on positive preference semantic matching, neglecting negative feedback signals and personalized collaborative information. V2 and V3 demonstrate substantial improvements by incorporating command-aware sequential modeling, where the AIA component effectively bridges explicit commands with implicit behavioral preferences. The progression from V3 to Full illustrates additive benefits of tool integration, where the Filter Tool provides additional gains through hard constraint enforcement and computational efficiency via candidate space pruning. These results validate the necessity of each component, demonstrating that optimal performance requires coordinated deployment of all modules to capture multifaceted user preferences in interactive contexts.

## 4.3 Online Experiments

*4.3.1* ***Long-term Performance.*** To investigate the long-term effectiveness of RecBot in real-world scenarios, we conducted comprehensive online A/B testing over a *three-month* period within a leading Asian e-commerce platform. As shown in Table 4, RecBot achieves a notable 0.71% reduction in Negative Feedback Frequency (NFF), validating that explicit linguistic guidance substantially enhances recommendation accuracy over implicit behavioral signals. Besides, RecBot effectively mitigates information cocoon effects with 0.88% and 1.44% improvements in Exposed Item Category Diversity (EICD) and Clicked Item Category Diversity (CICD). From a business perspective, the framework delivers 0.56%, 1.28%, and 1.40% improvements in Page Views (PV), Add-to-Cart (ATC), and Gross Merchandise Volume (GMV), respectively. These improvements in user experience and business metrics validate the commercial value of user-centric controllable recommendations.

*4.3.2* ***User-Command Fulfillment Analysis.*** To directly assess the real-world command fulfillment capabilities of the deployed RecBot system, we conducted a detailed evaluation using both human expert annotation and an *llm-as-a-Judge* approach (Qwen3-14B-Instruct fine-tuned on expert data). The evaluation focused on whether items in subsequent recommendation feeds strictly adhered to user-specified requirements following system policy adjustments, with each interaction classified as successful (complete

**Table 5: Performance Evaluation of Online RecBot's User Command Fulfillment. The LLM-Judge accuracy is validated at 96.5% using human evaluation as ground truth.**

| Evaluation Method | Success Rate (%) |
|---|---|
| Human Evaluation | 88.9 |
| LLM-Judge Evaluation | 87.5 |

requirement satisfaction) or failed (non-compliance). Our analysis encompassed approximately 180,000 online interaction instances. As presented in Table 5, the online RecBot achieved an 88.9% success rate according to human expert evaluation, while the automated LLM-Judge evaluation yielded a closely aligned 87.5% success rate with 96.5% consistency compared to human assessments. These results demonstrate that RecBot accurately interprets and responds to natural language user commands in production environments, while our LLM-based evaluation methodology provides a reliable and cost-effective alternative to manual assessment for large-scale commercial deployment monitoring.

*4.3.3* ***User Group Analysis.*** To investigate RecBot's differential impacts across different user populations, we conducted user group analysis by partitioning users based on *historical negative feedback frequency*. As shown in Fig. 6, RecBot demonstrates robust effectiveness across different user groups, achieving consistent NFF reductions ranging from 2.4% to 3.3% for users with low to moderately-high historical negative feedback frequencies. The [20,50) group shows optimal 3.3% improvement, indicating that natural language commands enable users with varying complaint patterns to articulate preferences more effectively than traditional implicit mechanisms. However, users with extremely high historical negative feedback [100,200) show a 1.6% increase in NFF, as these chronically dissatisfied users represent inherently challenging cases requiring comprehensive improvements beyond recommendation accuracy. For an illustrative case study, please refer to Appendix B.

# 5 Related Work

## 5.1 Interactive Recommendation

Interactive recommender systems enable dynamic adaptation to user preferences through real-time feedback mechanisms, typically formulated as Markov Decision Processes (MDP) where policies are iteratively updated based on user responses [6, 41, 44, 57]. Recent advances explore diverse optimization objectives: CIRS [7] addresses filter bubble effects through causal reasoning-enhanced offline reinforcement learning, while BiLLP [33] leverages LLMs' planning capabilities for long-term recommendation through bi-level learning. With large language model agents [39], conversational interactive recommendation has gained prominence through natural language interfaces. InstructAgent [48] introduces LLM-mediated frameworks for free-text instruction interpretation and knowledge-enhanced re-ranking, while InteRecAgent [16] combines LLMs as reasoning engines with traditional recommenders through customized toolsets. However, these approaches are constrained to dialogue-based assistant interfaces, limiting applicability to mainstream recommendation feeds where users lack direct communication channels. Our work addresses this gap by introducing the Interactive Recommendation Feed paradigm, enabling natural language commands within traditional feed-based interfaces.

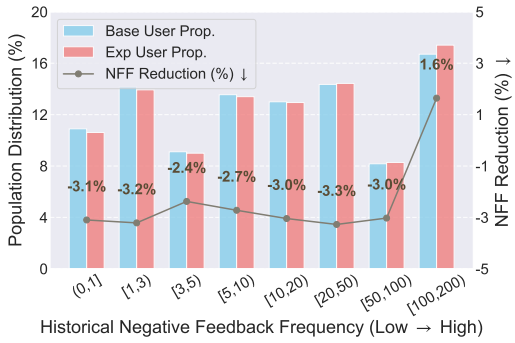



**Figure 6: Online performance improvements across different user groups split by historical negative feedback frequency.**

## 5.2 Controllable Recommender Systems

Controllable recommender systems enable dynamic adaptation to user preferences and platform objectives without model retraining [31]. Existing approaches focus on three dimensions: *Multi-objective Control* balancing competing goals through ComiRec [2] and CMR [3]; *User Portrait Control* enabling preference specification via LACE [25] and TEARS [28]; and *Scenario Adaptation Control* for cross-context variations in HyperBandit [32] and Hamur [20]. Technical solutions include hypernetwork-based parameter generation (CMR [3], HyperBandit [32]), Pareto optimization (MoFIR [9]), and LLM integration (RecLM-gen [24], LangPTune [8]). Control paradigms vary by stakeholder orientation: user-centric methods like UCRS [40] empower individuals, while platform-mediated solutions such as CCDF [54] focus on business objectives. Unlike these predefined control mechanisms, our method enables direct user communication through free-form natural language commands for real-time preference refinement and enhanced user control.

# 6 Conclusion

In this paper, we identify fundamental limitations in traditional recommender systems that constrain user expression through passive feedback paradigms, leading to systematic misalignment between user intentions and system interpretations. To address these challenges, we introduce the Interactive Recommendation Feed (IRF) paradigm and develop RecBot, a multi-agent framework that enables user-controllable recommendation experiences through natural language commands. Comprehensive evaluation across three datasets and long-term online deployment validate the effectiveness of our proposed approach, achieving substantial improvements in both user satisfaction and business outcomes.


# Acknowledgments

This work is supported in part by National Natural Science Foundation of China (No. 62472427 and No. 62422215), Beijing Outstanding Young Scientist Program NO.BJJWZYJH012019100020098, Intelligent Social Governance Platform, Major Innovation & Planning Interdisciplinary Platform for the "Double-First Class" Initiative, Renmin University of China, Public Computing Cloud, Renmin University of China, fund for building world-class universities (disciplines) of Renmin University of China, Intelligent Social Governance Platform. This work was supported by Alibaba Group through Alibaba Innovative Research Program.

## References


[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Yukuo Cen, Jianwei Zhang, Xu Zou, Chang Zhou, Hongxia Yang, and Jie Tang. 2020. Controllable multi-interest framework for recommendation. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2942–2951.

[3] Sirui Chen, Yuan Wang, Zijing Wen, Zhiyu Li, Changshuo Zhang, Xiao Zhang, Quan Lin, Cheng Zhu, and Jun Xu. 2023. Controllable multi-objective re-ranking with policy hypernetworks. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*. 3855–3864.

[4] Hernani Costa and Luis Macedo. 2013. Emotion-based recommender system for overcoming the problem of information overload. In *International conference on practical applications of agents and multi-agent systems*. Springer, 178–189.

[5] James Davidson, Benjamin Liebald, Junning Liu, Palash Nandy, Taylor Van Vleet, Ullas Gargi, Sujoy Gupta, Yu He, Mike Lambert, Blake Livingston, et al. 2010. The YouTube video recommendation system. In *Proceedings of the fourth ACM conference on Recommender systems*. 293–296.

[6] Chongming Gao, Kexin Huang, Jiawei Chen, Yuan Zhang, Biao Li, Peng Jiang, Shiqi Wang, Zhong Zhang, and Xiangnan He. 2023. Alleviating matthew effect of offline reinforcement learning in interactive recommendation. In *Proceedings of the 46th international ACM SIGIR conference on research and development in information retrieval*. 238–248.

[7] Chongming Gao, Shiqi Wang, Shijun Li, Jiawei Chen, Xiangnan He, Wenqiang Lei, Biao Li, Yuan Zhang, and Peng Jiang. 2023. CIRS: Bursting filter bubbles by counterfactual interactive recommender system. *ACM Transactions on Information Systems* 42, 1 (2023), 1–27.

[8] Zhaolin Gao, Joyce Zhou, Yijia Dai, and Thorsten Joachims. 2024. End-to-end Training for Recommendation with Language-based User Profiles. *arXiv preprint arXiv:2410.18870* (2024).

[9] Yingqiang Ge, Xiaoting Zhao, Lucia Yu, Saurabh Paul, Diane Hu, Chu-Cheng Hsieh, and Yongfeng Zhang. 2022. Toward pareto efficient fairness-utility trade-off in recommendation through reinforcement learning. In *Proceedings of the fifteenth ACM international conference on web search and data mining*. 316–324.

[10] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. 2024. A survey on llm-as-a-judge. *arXiv preprint arXiv:2411.15594* (2024).

[11] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. 2022. Towards universal sequence representation learning for recommender systems. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 585–593.

[12] Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. 2023. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. *arXiv preprint arXiv:2305.02301* (2023).

[13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.

[14] Jian Hu, Xibin Wu, Wei Shen, Jason Klein Liu, Zilin Zhu, Weixun Wang, Songlin Jiang, Haoran Wang, Hao Chen, Bin Chen, et al. 2024. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143* (2024).

[15] Jiani Huang, Shijie Wang, Liang-bo Ning, Wenqi Fan, Shuaiqiang Wang, Dawei Yin, and Qing Li. 2025. Towards Next-Generation Recommender Systems: A Benchmark for Personalized Recommendation Assistant with LLMs. *arXiv preprint arXiv:2503.09382* (2025).

[16] Xu Huang, Jianxun Lian, Yuxuan Lei, Jing Yao, Defu Lian, and Xing Xie. 2025. Recommender ai agent: Integrating large language models for interactive recommendations. *ACM Transactions on Information Systems* 43, 4 (2025), 1–33.

[17] Olivier Jeunen. 2023. A probabilistic position bias model for short-video recommendation feeds. In *Proceedings of the 17th ACM conference on recommender systems*. 675–681.

[18] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.

[19] Shah Khusro, Zafar Ali, and Irfan Ullah. 2016. Recommender systems: issues, challenges, and research opportunities. In *Information science and applications (ICISA) 2016*. Springer, 1179–1189.

[20] Xiaopeng Li, Fan Yan, Xiangyu Zhao, Yichao Wang, Bo Chen, Huifeng Guo, and Ruiming Tang. 2023. Hamur: Hyper adapter for multi-domain recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 1268–1277.

[21] Zhong-Zhi Li, Duzhen Zhang, Ming-Liang Zhang, Jiaxin Zhang, Zengyan Liu, Yuxuan Yao, Haotian Xu, Junhao Zheng, Pei-Jie Wang, Xiuyi Chen, et al. 2025. From system 1 to system 2: A survey of reasoning large language models. *arXiv preprint arXiv:2502.17419* (2025).

[22] Chia-Yu Lin, Li-Chun Wang, and Kun-Hung Tsai. 2018. Hybrid real-time matrix factorization for implicit feedback recommendation systems. *Ieee Access* 6 (2018), 21369–21380.

[23] Siyi Lin, Sheng Zhou, Jiawei Chen, Yan Feng, Qihao Shi, Chun Chen, Ying Li, and Can Wang. 2024. ReCRec: Reasoning the causes of implicit feedback for debiased recommendation. *ACM Transactions on Information Systems* 42, 6 (2024), 1–26.

[24] Wensheng Lu, Jianxun Lian, Wei Zhang, Guanghua Li, Mingyang Zhou, Hao Liao, and Xing Xie. 2024. Aligning large language models for controllable recommendations. *arXiv preprint arXiv:2403.05063* (2024).

[25] Sheshera Mysore, Mahmood Jasim, Andrew McCallum, and Hamed Zamani. 2023. Editable user profiles for controllable text recommendations. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 993–1003.

[26] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).

[27] Avinash Patil and Aryan Jadon. 2025. Advancing reasoning in large language models: Promising methods and approaches. *arXiv preprint arXiv:2502.03671* (2025).

[28] Emiliano Penaloza, Olivier Gouvert, Haolun Wu, and Laurent Charlin. 2024. TEARS: Textual Representations for Scrutable Recommendations. *arXiv preprint arXiv:2410.19302* (2024).

[29] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084* (2019).

[30] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.

[31] Chenglei SHEN, Xiao ZHANG, Teng SHI, Changshuo ZHANG, Guofu XIE, Jun XU, Ming HE, and Jianping FAN. 2025. A Survey of Controllable Learning: Methods and Applications in Information Retrieval. *Frontiers of Computer Science* (2025). doi:10.1007/s11704-025-41366-5

[32] Chenglei Shen, Xiao Zhang, Wei Wei, and Jun Xu. 2023. Hyperbandit: Contextual bandit with hypernewtork for time-varying user preferences in streaming recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 2239–2248.

[33] Wentao Shi, Xiangnan He, Yang Zhang, Chongming Gao, Xinyue Li, Jizhi Zhang, Qifan Wang, and Fuli Feng. 2024. Large language models are learnable planners for long-term recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1893–1903.

[34] Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Hanjie Chen, et al. 2025. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419* (2025).

[35] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.

[36] Jiakai Tang, Xu Chen, Wen Chen, Jian Wu, Yuning Jiang, and Bo Zheng. 2026. Parallel Latent Reasoning for Sequential Recommendation. *arXiv preprint arXiv:2601.03153* (2026).

[37] Jiakai Tang, Shiqi Shen, ZhipengWang ZhipengWang, Gong Zhi, Xueyang Feng, Zexu Sun, Haoran Tan, and Xu Chen. 2025. KAPA: A Deliberative Agent Framework with Tree-Structured Knowledge Base for Multi-Domain User Intent Understanding. In *Findings of the Association for Computational Linguistics: ACL 2025*. 6150–6166.

[38] Jiakai Tang, Chuan Wang, Gaoming Yang, Han Wu, Jiahao Yu, Jian Wu, Jianwu Hu, Junjun Zheng, Longbin Li, Shuwen Xiao, et al. 2025. ReaSeq: Unleashing World Knowledge via Reasoning for Sequential Modeling. *arXiv preprint arXiv:2512.21257* (2025).

[39] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.

[40] Wenjie Wang, Fuli Feng, Liqiang Nie, and Tat-Seng Chua. 2022. User-controllable recommendation against filter bubbles. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*. 1251–1261.

[41] Xinxi Wang, Yi Wang, David Hsu, and Ye Wang. 2014. Exploration in interactive personalized music recommendation: a reinforcement learning approach. *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)* 11, 1 (2014), 1–22.

[42] Yuling Wang, Changxin Tian, Binbin Hu, Yanhua Yu, Ziqi Liu, Zhiqiang Zhang, Jun Zhou, Liang Pang, and Xiao Wang. 2024. Can small language models be good reasoners for sequential recommendation?. In *Proceedings of the ACM Web Conference 2024*. 3876–3887.

[43] Wei Wei, Xubin Ren, Jiabin Tang, Qinyong Wang, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. 2024. Llmrec: Large language models with graph augmentation for recommendation. In *Proceedings of the 17th ACM international conference on web search and data mining*. 806–815.
[44] Yaxiong Wu, Craig Macdonald, and Iadh Ounis. 2021. Partially observable reinforcement learning for dialog-based interactive recommendation. In *Proceedings of the 15th ACM conference on recommender systems*. 241–251.
[45] Yaxiong Wu, Craig Macdonald, and Iadh Ounis. 2023. Goal-oriented multi-modal interactive recommendation with verbal and non-verbal relevance feedback. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 362–373.
[46] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. 2024. C-pack: Packed resources for general chinese embeddings. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*. 641–649.
[47] Fengli Xu, Qianyue Hao, Zefang Zong, Jingwei Wang, Yunke Zhang, Jingyi Wang, Xiaochong Lan, Jiahui Gong, Tianjian Ouyang, Fanjin Meng, et al. 2025. Towards large reasoning models: A survey of reinforced reasoning with large language models. *arXiv preprint arXiv:2501.09686* (2025).
[48] Wujiang Xu, Yunxiao Shi, Zujie Liang, Xuying Ning, Kai Mei, Kun Wang, Xi Zhu, Min Xu, and Yongfeng Zhang. 2025. Instructagent: Building user controllable recommender via llm agent. *arXiv e-prints* (2025), arXiv–2502.
[49] Wujiang Xu, Qitian Wu, Zujie Liang, Jiaojiao Han, Xuying Ning, Yunxiao Shi, Wenfang Lin, and Yongfeng Zhang. 2024. SLMRec: Distilling large language models into small for sequential recommendation. *arXiv preprint arXiv:2405.17890* (2024).
[50] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
[51] Chao Yi, Dian Chen, Gaoyang Guo, Jiakai Tang, Jian Wu, Jing Yu, Sunhao Dai, Wen Chen, Wenjun Yang, Yuning Jiang, et al. 2025. RecGPT Technical Report. *arXiv preprint arXiv:2507.22879* (2025).
[52] Chao Yi, Dian Chen, Gaoyang Guo, Jiakai Tang, Jian Wu, Jing Yu, Mao Zhang, Wen Chen, Wenjun Yang, Yujie Luo, et al. 2025. RecGPT-V2 Technical Report. *arXiv preprint arXiv:2512.14503* (2025).
[53] Zheng Yuan, Fajie Yuan, Yu Song, Youhua Li, Junchen Fu, Fei Yang, Yunzhu Pan, and Yongxin Ni. 2023. Where to go next for recommender systems? id- vs. modality-based recommender models revisited. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2639–2649.
[54] Tao Zhang, Luwei Yang, Zhibo Xiao, Wen Jiang, and Wei Ning. 2024. On practical diversified recommendation with controllable category diversity framework. In *Companion Proceedings of the ACM Web Conference 2024*. 255–263.
[55] Xiaoyu Zhang, Ruobing Xie, Yougang Lyu, Xin Xin, Pengjie Ren, Mingfei Liang, Bo Zhang, Zhanhui Kang, Maarten de Rijke, and Zhaochun Ren. 2024. Towards empathetic conversational recommender systems. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 84–93.
[56] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.
[57] Sijin Zhou, Xinyi Dai, Haokun Chen, Weinan Zhang, Kan Ren, Ruiming Tang, Xiuqiang He, and Yong Yu. 2020. Interactive recommender system via knowledge graph-enhanced reinforcement learning. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*. 179–188.
[58] Yongchun Zhu, Jingwu Chen, Ling Chen, Yitan Li, Feng Zhang, and Zuotao Liu. 2024. Interest clock: Time perception in real-time streaming recommendation system. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2915–2919.

# A Experimental Details

## A.1 Offline Interactive Recommendation Experiments

To closely simulate realistic online user-system interaction environments, we employ GPT-4.1[1] as the backbone for our user simulation framework, leveraging its robust language generation capabilities to emulate authentic user behaviors. Based on prevalent user command patterns observed in production systems, we design three distinct interaction scenarios of increasing complexity:

[1]The specific snapshot version used is GPT-4.1-2025-04-14.

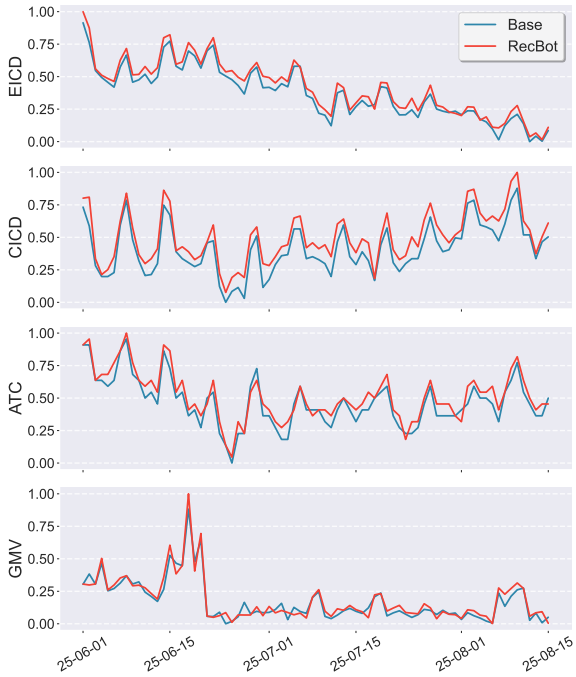



**Figure 7: Online performance curves during the three-month A/B testing period. The comparison shows RecBot *vs.* the base system with all metrics normalized using min-max scaling.**

- **Single-Round Interaction (SR):** Users possess clearly defined preferences and can express their requirements for target item characteristics comprehensively within a single command. This scenario is suited for users who have clear intentions.
- **Multi-Round Interaction (MR):** Users initially exhibit ambiguous or exploratory preferences. Through iterative exposure to recommendation feeds, users progressively refine their requirements by providing both positive interest signals for desired attributes and negative feedback regarding unsatisfactory aspects of current recommendations. In our implementation, we employ pre-constructed prompting strategies to drive diverse user feedback styles, enabling both proactive new requirements and reactive negative feedback to exposed items. This iterative refinement process expects the system to dynamically adjust its recommendation policy based on accumulated user guidance.
- **Multi-Round Interaction with Interest Drift (MRID):** Building upon the MR scenario, this setting introduces preference evolution where users may exhibit conflicting or shifting interests across interaction rounds. For instance, a user initially seeking Windows-based computers may subsequently pivot toward Mac preferences during the browsing session. This scenario represents the most challenging yet realistic user behavior pattern commonly observed in exploratory contexts. In our implementation, we simulate this interest drift by initially pre-sampling a random item as a pseudo-target to guide the user's early interactions, then strategically redirecting the user's preference toward their actual ground-truth next interaction item at round 3, thereby creating a realistic preference shift scenario.




**Figure 8: Case study of RecBot on production platform.**

For multi-round interaction scenarios (*i.e.*, MR and MRID), we establish experimental parameters with a maximum interaction limit of 5 rounds and recommendation list size of $K = 5$ items per round, reflecting typical user attention spans focused on top-ranked recommendations [17]. We assume that users will select the top-1 recommended item from each exposed list to issue explicit feedback commands that guide the system's strategy adjustment for subsequent rounds. The interaction terminates when either: (1) the user's ground-truth target item appears within the top-5 recommendations, indicating successful preference satisfaction, or (2) the maximum round limit is exceeded, representing interaction failure or user abandonment. These different experimental designs enable systematic evaluation of RecBot's adaptive capabilities across diverse user interaction patterns while maintaining computational feasibility and realistic user behavior modeling.

## A.2 Evaluation Metrics

For offline experiments, we follow prior work [40, 42] and adopt ranking metric **Recall@N**, where $N \in \{10, 20, 50\}$. Additionally, we introduce three specialized metrics to comprehensively evaluate interactive recommendation performance:

- **Condition Satisfaction Rate (CSR@N):** This metric measures the attribute-level coverage ratio between recommended items in top-$N$ positions and the target item's characteristics. CSR@N serves as an attribute-oriented ranking accuracy measure that, compared to the strict precision requirements of Recall and NDCG, reflects the model's capability to correctly infer user preferences regarding item category and attribute preferences.
- **Pass Rate (PR):** This binary metric calculates whether the target item is successfully delivered within the top-$K$ recommendation feed during the limited interaction rounds $T$ ($T$=1 for SR scenarios, $T$=5 for MR and MRID scenarios).
- **Average Rounds (AR):** This efficiency metric quantifies the average interaction rounds required for the system to successfully predict the target item to users at the top-$K$ positions. For cases where the system fails to recommend the target item within the maximum $T$ rounds, we assign a penalty score of $T$+1 to reflect unsuccessful interactions.

For online experiments, we assess RecBot's real-world performance through two complementary evaluation dimensions that capture both user satisfaction and business impact. In specific, **(1) User Experience Metrics** focus on measuring user satisfaction and engagement quality, including: **Negative Feedback Frequency (NFF)**, measuring the frequency of user dissatisfaction signals toward recommendation feeds; **Exposed Item Category Diversity (EICD)**, quantifying the average category diversity of items presented to users; **Clicked Item Category Diversity (CICD)**, measuring the average category diversity of items users actively engage with through clicks. **(2) Business Utility Metrics** evaluate the commercial utility, including **Page Views (PV)**, capturing overall user engagement with recommended content; **Add-to-Cart (ATC)**, measuring conversion intent through shopping cart additions; **Gross Merchandise Volume (GMV)**, quantifying the total transaction value generated from recommendations.

## B Case Study

To illustrate RecBot's practical effectiveness, we present a representative multi-round interaction case from our production deployment as shown in Fig. 8. The user initially receives short skirts but requests "long skirts" for autumn weather, prompting RecBot to adapt with longer garments. Across subsequent rounds, the user progressively adds constraints: color preference ("light blue"), budget limitation ("around 200"), and negative feedback ("Don't want floral dresses"). RecBot successfully maintains all accumulated preferences while satisfying each new requirement, demonstrating effective command parsing, memory consolidation, and adaptive tool orchestration. The interaction achieves successful convergence in Round 4 with complete user satisfaction ("Perfect, I love this long dress!"), exemplifying how natural language commands enable efficient preference refinement.